

Discriminative and Generative Neuronal Networks on FSD Break Data

Chenwei Li, Jingyao Zhang, Moritz, Xuliang Xu

June 23, 2025

1 Introduction

The coding project of the Neural Network Seminar thought by Marina Dolokov in 2025 is about processing data of break measurements collected by the company FSD in Radebeul. The project consists out of two groups tackling different aspects of deep learning in the context of the given data. One group of students working on the bias in the dataset to enhance the learning process. The second group tries to create a generative model to increase the set on which further models can be trained. In this short overview, we present the thought, problems and results of our group working on the generative AI task although also in this group, preprocessing of the data plays a crucial role.

1.0.1 Dataset and Task

The dataset consists of multi-channel time series sensor data, stored in pickle files (`train.pickle`, `test.pickle`). Each sample includes a sequence of acceleration and rotation sensor readings in x,y,z-directions and a corresponding class label. The goal is to build a neural network model that accurately classifies the braking state. We also try to generate new data to enhance the classifiers performance.

Contents

1	Introduction	1
2	Task 1: FCN, 2D CNNs vs. LSTMs Parameter Tests & Comparison	3
2.1	FCN: Fully Connected Network	3
2.2	2D CNN	9
2.3	LSTM: Long Short-Term Memory Model	13
3	Task 2: Generative AI	16
3.1	GAN	16
3.2	RNN	17
3.3	cGAN	18
3.4	Transformer-GAN	19
3.5	DANN	20
3.6	Result Analysis	20
3.7	Dataset Expansion and Performance Improvement	25
4	Comparison of different models	26
5	Additional Task 2: Merging Models	28

2 Task 1: FCN, 2D CNNs vs. LSTMs Parameter Tests & Comparison

2.1 FCN: Fully Connected Network

(Jingyao Zhang)

Sensor data classification is a critical task in analyzing time-series data for various applications such as fault detection and predictive maintenance. This section documents our efforts to improve Fully Convolutional Network (FCN) models for sensor data classification. The research includes experiments with different methods, pipelines, and data preprocessing techniques to enhance model performance. Notably, we explored the use of Generative Adversarial Networks (GANs) for data augmentation and label simplifications.

2.1.1 Model Architecture

This Fully Convolutional Network (FCN) model architecture implemented for time-series or sequence classification tasks. The FCN leverages 1D convolutional layers to extract hierarchical features from sequential data, followed by global average pooling and a fully connected layer for classification.

The FCN model is defined as follows:

- **Input:** $x \in \mathbb{R}^{C \times L}$, where C is the number of input channels and L is the sequence length.
- **Conv1D Layer 1:** Applies a 1D convolution with 64 filters and kernel size 8, followed by a ReLU activation.
- **Conv1D Layer 2:** Applies a 1D convolution with 128 filters and kernel size 5, followed by a ReLU activation.
- **Conv1D Layer 3:** Applies a 1D convolution with 64 filters and kernel size 3, followed by a ReLU activation.
- **Global Average Pooling:** Reduces the sequence dimension to a single value per channel, resulting in a feature vector of length 64.
- **Dropout:** A dropout layer with dropout probability $p = 0.3$ for regularization.
- **Fully Connected Layer:** A linear layer mapping from 64 features to the number of classes.

2.1.2 Pipeline Workflow

The FCN pipeline workflow consists of multiple steps aimed at preprocessing, balancing, and standardizing data to make it suitable for training:

1. Data Loading

- Load raw training and testing data from pickle files.

- Extract `sensor_data` and `label` fields from the data.

2. Label Preprocessing

- Remap original labels to two categories:
 - Working (0)
 - Non-working (1)
- Print class distribution to check balance.

3. Data Normalization

- Apply z-score normalization using the training set's mean and standard deviation:

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}} + \epsilon}$$

- Ensure test data is normalized using the same statistics.

4. Data Reshaping

- Convert data to shape (batch, channels, timesteps) suitable for 1D CNN input.

5. Dataset and DataLoader Preparation

- Wrap data in a custom `BrakeDataset` class.
- Split training data into training and validation sets (e.g., 80%/20%).
- Use `PyTorch DataLoader` for efficient batch processing.

6. Model Initialization

- Instantiate the FCN model with the appropriate number of input channels and output classes.
- Move the model to the computation device (`cuda` or `cpu`).

7. Class Weight Calculation

- Compute class weights to address class imbalance:

$$w_j = \frac{n}{k \cdot n_j}$$

where n is the total number of samples, k is the number of classes, and n_j is the number of samples in class j .

- Pass the class weights to the loss function.

8. Training

- Train the model for a fixed number of epochs using `CrossEntropyLoss` (with optional class weighting).
- Monitor loss and validation accuracy after each epoch.

9. Testing

- Evaluate the trained model on the test set.
- Report the final test accuracy.

		normalization	
#Trial		1	2
data	MaxAbs Scaling	Maxabs	
	z_score standardize		z_score
RESULT	Val_Acc	0.0444	0.9408
	Test_Acc	0.1049	0.3636

Figure 1: Normalization

2.1.3 Experiments and Evolution

In order to optimize the Fully Convolutional Network (FCN) for our classification task, we conducted a series of experiments focusing on several key variables. The variables explored included both data processing strategies and model/training configurations. The following figure summarizes the main factors considered:

1. Experimental Variables

- **Data Processing:**

- **Normalization:** Compared MaxAbs scaling and z-score standardization.
- **Number of Categories:** Tested performance on both binary (2) and multi-class (12) classification.
- **Data Augmentation:** Investigated traditional upsampling techniques for class balance, as well as generative approaches such as GAN, cGAN, tGAN, and RNN-based augmentation.

- **Model and Training:**

- **Regularization:** Explored L2 regularization and different dropout rates (0.1, 0.3, 0.5).
- **Batch Size:** Evaluated batch sizes of 32, 64, and 128.
- **Split Strategy:** Compared random split and K-Fold cross-validation.
- **Optimizer:** Tested Adam optimizer with learning rates of 0.001, 0.0005, and 0.005.
- **Loss Function:** Used CrossEntropyLoss with and without class weights to address class imbalance.
- **Epochs:** Analyzed training for 20, 30, and 40 epochs.
- **Early Stopping:** Implemented early stopping with patience to prevent overfitting.

2. Key Experimental Results

For each variable above, systematic experiments were conducted. The key conclusions for each are summarized below.

- **Normalization:**
- **Category Number:**
- **Data Augmentation:**
- **Split Strategy:**

		normalization: class	
		2	3
data	MaxAbs Scaling		
	z_score standardize	z_score	y
classes	12/2 (remap)	12	2
Data augmentation	upsampling minority classes		
	Gan1		
	Gan2		
	cGAN		
	dann		
	rnn		
	tGAN		
FCN model	dropout(p)	0.3	0.3
	early_stopping(patience)		
split?	random split		
	K-Folds		
Batch_size		64	64
optimizer(adar)	lr		
	L2()		
loss fct(CrossE	Compute class weights		y
Epochs			30
RESULT	Val_Acc	0.9408	0.9938
	Test_Acc	0.3636	0.6434

Figure 2: FCNCategories

		gan1(75%)	gan2(100%)	cgan(175%)	dann(120%)	rnn(200%)	gan1(75%)	gan2(100%)
RESULT	Val_Acc	0.9889	0.7762	0.9911	0.9423	0.9907	0.7762	0.9907
	Test_Acc	0.6673	0.6876	0.6514	0.6462	0.6118	0.6468	0.6344

Figure 3: FCNDataAugmentation

		batch	split	
		5	6	
data	MaxAbs Scaling			
	z_score standardize	y	y	y
classes	12/2 (remap)	2	2	2
Data augmentation	upsampling minority classes			
	Gan1			
	Gan2			
	cGAN			
	dann			
	rnn			
	tGAN			
FCN model	dropout(p)	0.3	0.3	0.3
	early_stopping(patience)			
split?	random split		5(22min)	
	K-Folds			k-Folds
Batch_size		64	64	64
optimizer(adar)	lr			
	L2()			
loss fct(CrossE	Compute class weights	y	y	y
Epochs		30	30	30
RESULT	Val_Acc	0.9889	0.9938	0.9931
	Test_Acc	0.6673	0.6414	0.6344

Figure 4: FCNSplitStrategy

2.1.4 Result Analysis

- **Normalization:** z-score standardization provided more stable and improved accuracy compared to MaxAbs scaling.
- **Category Number:** Binary classification (2 categories) achieved higher performance and was less prone to overfitting than the 12-category setup.
- **Data Augmentation:** Generative methods (GAN-based) outperformed traditional upsampling for handling class imbalance, especially when the minority class was underrepresented.
- **Regularization:** Dropout of 0.3 and L2 regularization (0.01) gave the best generalization. Too high dropout (0.5) led to underfitting.
- **Batch Size:** Batch size of 64 balanced performance and convergence speed.
- **Split Strategy:** K-Fold cross-validation ensured more robust evaluation than random split.
- **Optimizer:** Adam with a learning rate of 0.001 gave the most consistent results.
- **Loss Function:** Incorporating class weights in CrossEntropyLoss improved minority class recall.
- **Epochs:** 30 epochs were generally sufficient for convergence with early stopping.
- **Early Stopping:** Early stopping effectively prevented overfitting, especially in longer runs.

2.1.5 Extended Model:

In addition to the FCN architecture, we also experimented with a Multi-Layer Perceptron (MLP) model. The MLP consists of two hidden layers with dropout regularization and uses softmax activation in the output layer. Despite its simple structure, it achieved results comparable to the best-performing models in our study.

The Multi-Layer Perceptron (MLP) model takes the input feature vector and processes it through two fully connected layers, each followed by a ReLU activation and dropout for regularization. The final layer outputs class probabilities using a softmax activation. This straightforward feedforward structure allows the model to learn complex, non-linear relationships in the data while helping to prevent overfitting through dropout.

2.1.6 Conclusion and Future Work

1. Conclusion:

- Training with the original dataset, without augmentation or generative methods, yielded the best validation and test accuracy, indicating the dataset’s high quality for this task.
- Consistent use of z-score standardization contributed to stable training and faster convergence.

- While generative augmentation methods like cGAN and RNN showed some promise, they did not outperform the original data.

Overall, careful preprocessing and straightforward training strategies provided superior results compared to complex augmentation approaches.

2. Future Work:

- Future improvements could focus on reducing information loss by exploring multi-class or multi-label classification with more granular labels.
- Enhancing training stability and robustness through alternative loss functions, architectures, and training schedules is recommended.
- Additional strategies, such as advanced regularization, new augmentation methods (e.g., mixup, SMOTE), and hyperparameter tuning, may further optimize performance.

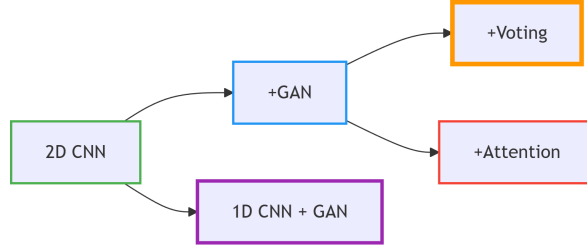


Figure 5: Model Architecture Spectrum

2.2 2D CNN

(Chenwei Li)

Convolutional Neural Networks (CNNs) exploit local connectivity and weight sharing to efficiently learn hierarchical features from grid-structured inputs. A 2D CNN applies convolutional kernels over spatial dimensions, making it well suited for extracting patterns from two-dimensional sensor representations (e.g., spectrogram-like matrices).

2.2.1 Model Variants & Rationale

1. 2D CNN (Baseline)

- **Architecture:** Treats sensor data as a $1 \times 128 \times 6$ image. Three $\text{Conv2D} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}$ blocks, followed by $\text{GlobalAveragePooling} + \text{Dense} \rightarrow \text{Softmax}$.
- **Purpose:** Simple baseline for spatial feature extraction.
- **Test Accuracy:** $\tilde{0.51}$ – 0.57 (limited generalization).

2. 2D CNN + GAN

- **Addition:** Incorporates GAN-generated synthetic samples into training data.
- **Purpose:** Augment minority fault classes.
- **Test Accuracy:** $\tilde{0.57}$ – 0.64 (medium stability, sensitive to GAN quality).

3. 2D CNN + GAN + Random Split Voting (Ensemble)

- **Description:** Trains multiple 2D CNN+GAN models on different random splits; aggregates predictions via hard or soft voting.
- **Purpose:** Reduce variance and enhance robustness.
- **Test Accuracy:** Consistently ≥ 0.60 (best and most stable).

4. 2D CNN + GAN + Multi-Head Attention

- **Description:** Adds multi-head self-attention after convolution blocks to weight feature positions.
- **Purpose:** Focus on critical time-channel patterns.
- **Test Accuracy:** $\tilde{0.50}$ – 0.60 (unstable, often worse than baseline).
- **Failure Analysis:**

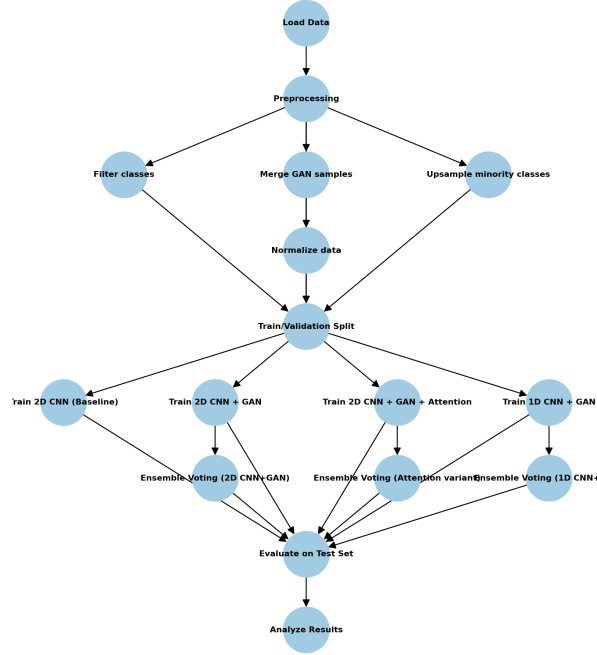


Figure 6: workflow

- Brake faults depend on local temporal spikes; global attention may mis-weight signals.
- Shallow CNN backbone limits feature abstraction for effective attention.

5. 1D CNN + GAN

- **Description:** Applies 1D convolutions along the time axis with GAN-augmented data.
- **Purpose:** Directly capture temporal patterns with fewer parameters.
- **Test Accuracy:** $\tilde{0.59}$ – 0.63 (stable, slightly below ensemble).

2.2.2 Workflow Overview

- **Data:** `train.pickle`, `generated_data.pickle`, `test.pickle`.
- **Preprocessing:** Filter classes, merge GAN data, upsample minority classes, normalize with training mean/std.
- **Training:** Adam optimizer, CrossEntropyLoss with class weights, ReduceLROnPlateau, EarlyStopping.
- **Evaluation:** Test accuracy, F1 scores, confusion matrices.

2.2.3 Results & Analysis

1. Performance Comparison
2. Test Accuracy Visualization

Insight: The ensemble model (2DCNN+GAN+Vote) consistently outperforms others, while the attention variant shows high variability.

Model	Test Accuracy Range	Stability	Key Observations
2D CNN + GAN + Vote	bigger than 0.60	Medium	Best stability & performance
1D CNN + GAN	0.59–0.63	High	Stable, outperforms 2D baseline
2D CNN + GAN	0.57–0.64	Medium	Sensitive to GAN quality
2D CNN (baseline)	0.51–0.57	High	Limited generalization
2D CNN + GAN + Multi-Head Attn	0.50–0.60	Low	Attention unstable, often v

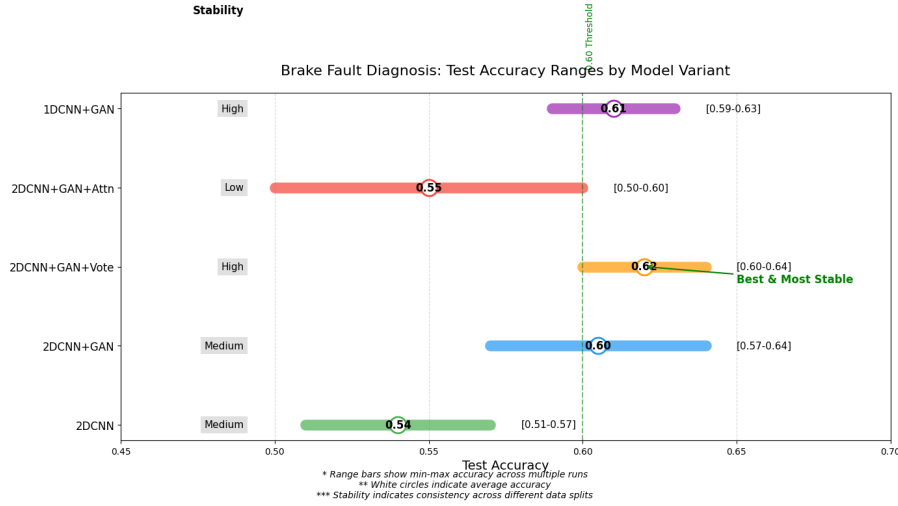


Figure 7: Test Accuracy

2.2.4 Core Issues & Diagnosis

- **Class Imbalance:** Minority classes (e.g., 1,2,4,5) have recall $\downarrow$ 0.35; "no-fault" class recall 0.96, skewing predictions.
- **Attention Failure:** Global attention mis-weights local temporal spikes critical to brake faults; shallow CNN limits abstraction.
- **Overfitting & Shift:** Validation accuracy $\downarrow$ 95% vs. test $\tilde{60\%}$; train/validation embeddings diverge from test data.

2.2.5 Conclusions & Future Work

1. Core Conclusions

- **Ensemble Success:** 2D CNN + GAN + Voting achieves $\downarrow$ 0.60 test accuracy with high stability.
- **Attention Ineffective:** Multi-head attention degrades performance in shallow CNNs.
- **Data Quality Key:** GAN samples must align with real distributions.
- **1D CNN Viable:** Stable at $\tilde{0.59-0.63}$, suits temporal data.

2. Future Work

- Optimize GANs with physical constraints and domain-adversarial training.
- Explore hybrid 1D+2D models or lightweight attention (e.g., SE blocks).

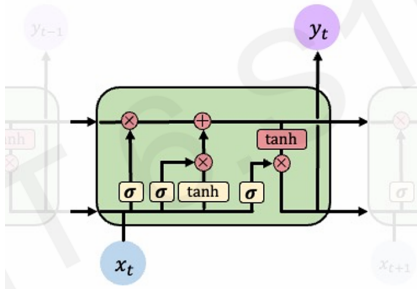
- Apply advanced augmentation (time-warping, noise injection).
- Use stratified k-fold CV and monitor per-class metrics.
- Enhance ensembles with diverse architectures.

2.3 LSTM: Long Short-Term Memory Model

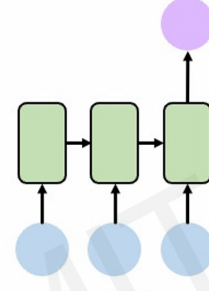
(Moritz Bauer)

2.3.1 Model Architecture

We recall the scheme of the lecture of LSTM networks.



(a) A single LSTM-Cell



(b) Multiple Iterations of the Dataset to get a classification

Since we use the acceleration and velocity measurements of one car with corresponding dimension (128, 6), we have 128 LSTM-cells expecting 6 data points each. Every cell is predicting a break condition code but only the last prediction is taken into account. The model is implemented in PyTorch. Further specifications:

- Hidden LSTM layer is fully connected to output layer
- Random-Split Validation of test and validation data is renewed every epoch
- Training data is shuffled before training
- use of Adam Optimizer
- use of Binary Cross Entropy Loss-Function

2.3.2 Pipeline Workflow

We used quite some time to improve the discriminative models, since the results haven't been too satisfying. The outlines of the process for the LSTM model are given below:

1. Label conversion and normalization of data

- We simplified the label to train our model with, we only differ by working/defect breaks by using the map for label x

$$x \mapsto \begin{cases} 1 & \text{if } x = 0 \\ 0 & \text{if } x > 0 \end{cases} \quad \text{later: } x \mapsto \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \end{cases}$$

- And for the purpose of easier detection, we normalized the data for the velocity and rotation measurement each by

$$AX_{norm} = (\frac{ax_1}{ax_{max}}, \dots, \frac{ax_{128}}{ax_{max}}), \quad ax_{max} = \max\{|a_1|, \dots, |a_{128}|\}$$

so we do not shift the data but ensure the maximal absolute value to be 1.
In fact, both methods turn out to hinder the training process. Especially our label conversion caused antilearning (significant bad results), since we have

label $x = 0$ (break defect) $\rightarrow$ LSTM needs to detect irregularity
label $x = 1$ (break in order) $\rightarrow$ LSTM needs to detect regularity

but somehow the model expects to detect any “property” for label $x = 1$. To solve this, we turned around the images of the map.

2. Bias in the Data

- Since our training data exists out of around 40% working breaks and 60% defect breaks, our model often learns to predict “This is a broken break” since it succeeds in 60% of the cases.
- To prevent this, we use weights corresponding to the label distribution on the loss function to punish our model harder if it falsely predicts a working break

3. Hard coded data extension

- Since our test dataset only exists out of around 4000 measurements with varying models and break conditions, the amount of examples the model has per case (e.g. model & break condition) is limited.
- To extend the data, we shifted data in time and approximated up to 5 new time steps or scaled the data by small factors of 0.95-1.05. This is to force the model to focus on the relevant information into the data.

We compare the development of the accuracy from step 1-3 under the following conditions:

Number and Dimension of Hidden Layer	1 x 512
Batch Size	32
Number and Size of Epochs	3 x 20
Ratio Test/Validation Data	3/1
Learning Rate	10^{-3}

Setting	Using 1	Not Using 1	Using 2	Using 2&3
Total accuracy	0.4	0.52	0.56	0.51
Working break accuracy	0	0.49	0.84	0.61
Broken break accuracy	1	0.67	0.36	0.36

At first place, hard coded data extension does not increase our models’ performance. If we plot the loss over the training process, the training seems to be quite unstable. To stabilize our training process, we now try to gather 64 measurements in one batch. We keep our weighted loss function and test different initial learning rates for the Adam optimizer:

Setting	$lr = 10^{-1}$, $b = 64$	$lr = 10^{-2}$, $b = 64$	$lr = 10^{-4}$, $b = 64$	$lr = 10^{-5}$, $b = 64$
Total accuracy	0.68	0.68	0.57	0.49
Working break accuracy	0.94	0.93	0.57	0.25
Broken break accuracy	0.3	0.33	0.59	0.84

2.3.3 Task 2.1

Since an initial learning rate of $10^{-1}/10^{-2}$ seemed to be the best option, we now try to train the model with learningrate $lr = 5 \cdot 10^{-2}$, augmented datasets (generated/hard coded), weights, and without normalization of the sensor data:

Dataset	Hard Coded	GAN	RNN	CGAN	TGAN	DANN
Total accuracy	0.69	0.7	0.68	0.64	0.69	0.67
Working break accuracy	0.87	0.89	0.83	0.89	0.95	0.87
Broken break accuracy	0.43	0.43	0.47	0.28	0.30	0.38

2.3.4 Result Analysis

The interpretation of the observed behaviour could be, that by normalizing the data, we mostly lose information instead of helping the model to increase its prediction. Setting weights is crucial to work against the biased training data and data extensions seems really helpful under the condition of a well chosen hyperparameter setting.

It seemed to be essential to give a sufficient large step-width to the Adam optimizer to let him “stray around” a bit to find good minima to approach against. Also observed was, that increasing the layers size and the models’ depth does not necessarily increase the models’ performance on the test data and only leads to longer coffee breaks.

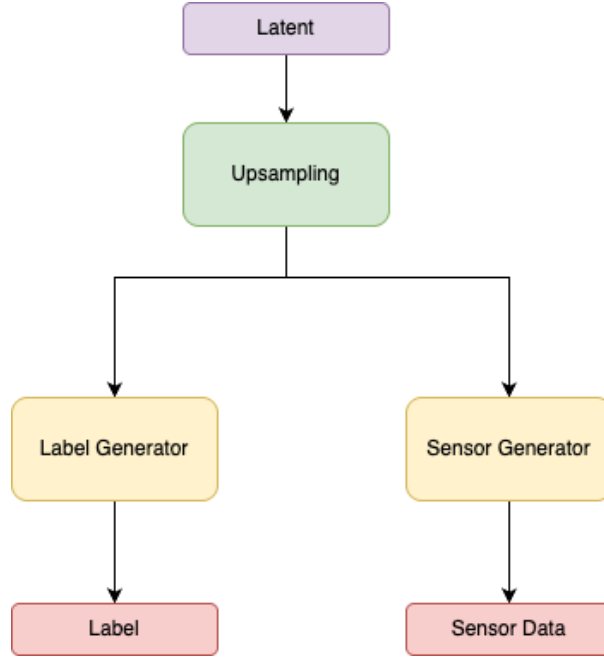


Figure 9: GAN Generator

3 Task 2: Generative AI

(Xuliang Xu)

Our group focused on Generative AI tasks. We initially implemented basic data generation using GAN and RNN models. To address training set label imbalance and distribution mismatches between training and test sets, we subsequently developed cGAN and DANN models. Finally, to capture more complex temporal patterns, we implemented transformer-based GANs.

3.1 GAN

Generative Adversarial Networks (GANs) are unsupervised learning methods involving two competing neural networks. The generator takes random latent space samples as input and produces outputs that mimic real training samples. The discriminator receives either real samples or generator outputs, aiming to distinguish between them. Through adversarial training, both networks collaborate to generate indistinguishable synthetic data.

3.1.1 Model Architecture

Both generator and discriminator are implemented in PyTorch with the following structure:

- Fully Connected Neural Networks (FCNN) serve as the base architecture
- Generator uses BatchNorm1d for stability with ReLU activation
- Discriminator employs Dropout regularization with LeakyReLU activation

3.1.2 Experiments and Evolution

Initially, the generator and discriminator had similar architectures, but the discriminator converged faster to a loss of 0.01. To achieve balanced adversarial training, we implemented:

- Increased generator capacity with more neurons

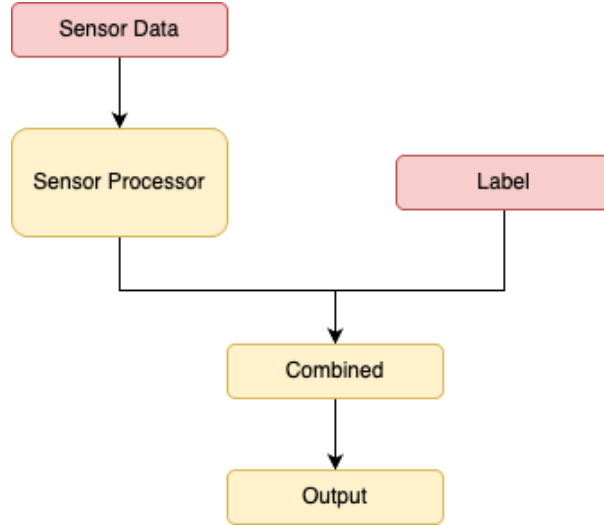


Figure 10: GAN Discriminator

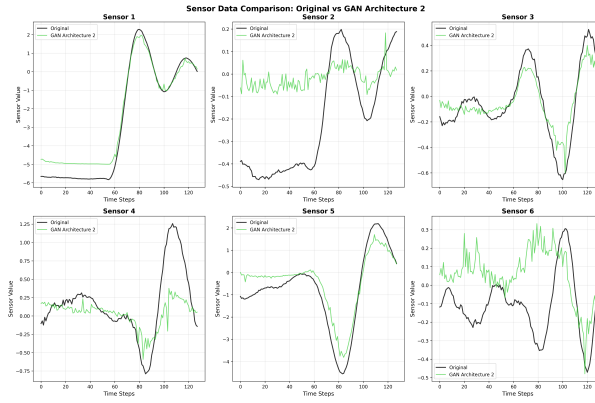


Figure 11: Data Comparison GAN

- Applied $5\times$ higher learning rate for the generator
- Provided additional training iterations for the generator
- Used soft labels (0.9/0.1) instead of hard labels (1/0) for smoother learning

These modifications stabilized discriminator loss around 0.7, indicating optimal equilibrium where the discriminator cannot reliably distinguish real from synthetic data. The soft labeling technique proved most impactful among these adjustments.

3.1.3 Pipeline Workflow

For Task 2, we employed a distinct preprocessing approach: converting original labels into four binary sub-labels represented by multi-head outputs, enabling the model to extract more generalizable features from the data.

3.2 RNN

Recurrent Neural Networks (RNNs) are well-suited for sequential data processing, making them ideal for simulating sensor time series data. Unlike fully connected networks, RNNs employ recurrent connections where neuron outputs from one time step serve as inputs for the next, enabling the capture of temporal dependencies and sequential patterns.

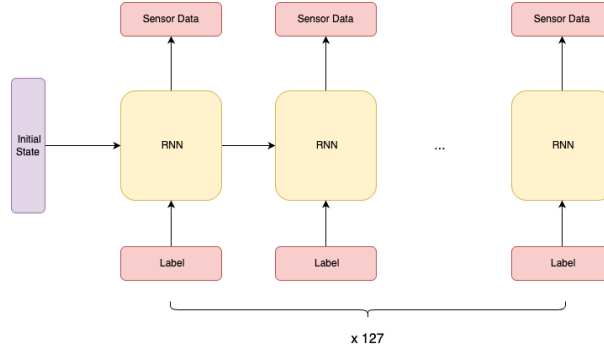


Figure 12: RNN

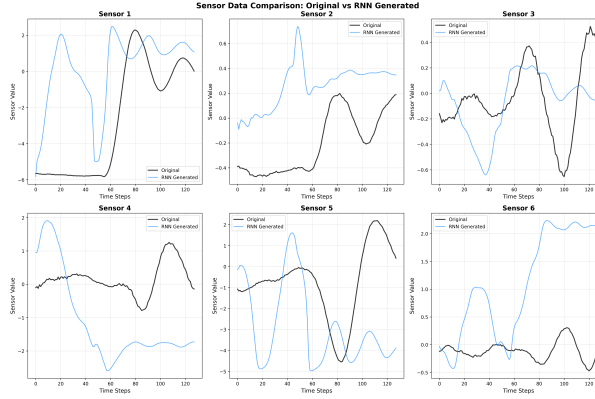


Figure 13: Data Comparison RNN

3.2.1 Model Architecture

- Input: sensor data from previous layer and 4 sub-labels
- Hidden layers: 5 layers with 128-dimensional hidden states
- Sequence length: 127 time steps

3.2.2 Experiments and Evolution

During training, the RNN loss consistently plateaued around 0.14 without meaningful reduction. Despite experimenting with various configurations—different hidden layer counts, dimensions, and sequence lengths—the training loss remained stable at approximately 0.14, indicating potential convergence limitations.

3.3 cGAN

Conditional Generative Adversarial Networks (cGANs) extend traditional GANs by incorporating conditional information into both generator and discriminator networks. Unlike standard GANs that generate samples from random noise, cGANs accept both latent vectors and conditional labels as input, enabling controlled generation of specific data types.

3.3.1 Model Architecture

- Implemented using Fully Connected Neural Networks (FCNN)

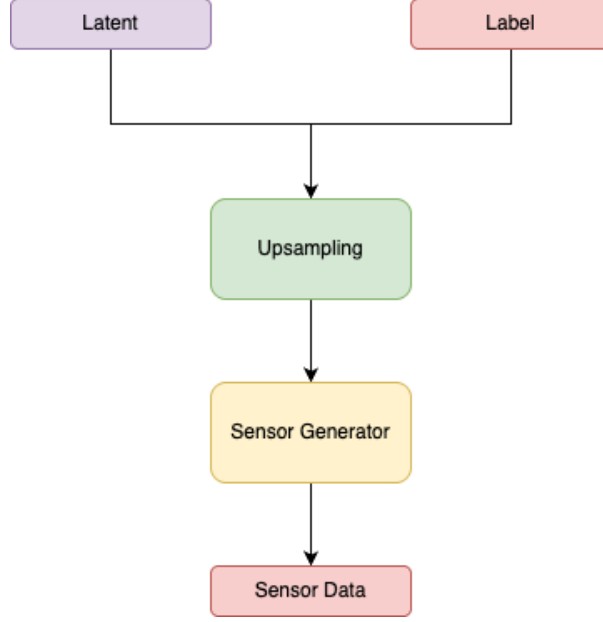


Figure 14: cGAN Generator

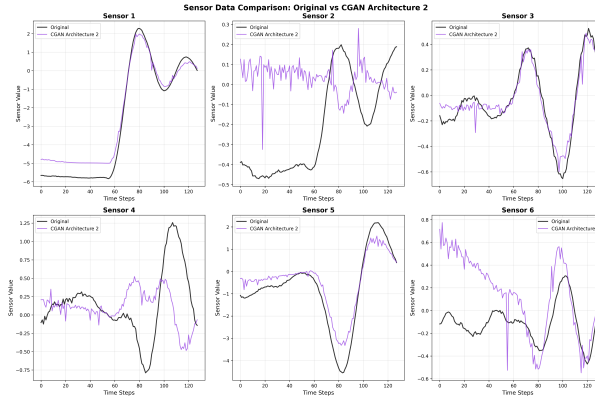


Figure 15: Data Comparison cGAN

- Generator employs LeakyReLU activation (unlike vanilla GAN's ReLU)

3.3.2 Experiments and Evolution

Initial cGAN training exhibited significant instability. Unlike vanilla GANs, the discriminator loss oscillated wildly between 0.01 and 100, rather than converging to small values. To stabilize training and maintain discriminator loss near 0.7, we implemented additional techniques beyond standard GAN methods:

- Switched generator activation to LeakyReLU
- Increased dropout rates in the discriminator for enhanced regularization

3.4 Transformer-GAN

Transformers are deep learning architectures utilizing attention mechanisms. To capture more complex temporal relationships and cross-sensor patterns, we integrated Transformer architec-

ture into GAN frameworks, investigating whether this approach could achieve superior simulation results.

3.4.1 Model Architecture

- Multi-sensor processing using Transformer encoder and decoder with 6 attention heads and 3 layers
- Learnable positional embeddings to capture temporal relationships in 128-length sequences
- Flattened feature representations for downstream tasks or network component integration

3.4.2 Experiments and Evolution

Leveraging experience from previous GAN implementations, we successfully stabilized the model training. However, the Transformer architecture demanded significantly higher computational resources, requiring over double the GPU memory and substantially longer training times compared to previous methods.

3.5 DANN

We observed that models absent from the training set appeared in the test set, reflecting real-world scenarios. To address this domain shift challenge, we employed Domain-Adversarial Neural Networks (DANN), which share architectural similarities with GANs but target domain adaptation problems. DANN uses a Label Predictor to classify features from a feature extractor, while a Domain Discriminator determines whether features originate from source or target domains, enabling extraction of domain-invariant representations.

3.5.1 Model Architecture

- Three-component architecture: FCNN Feature Extractor (1024→768→512 dimensions), Label Predictor, and Domain Classifier with adjustable gradient reversal
- Domain-Invariant Generator accepts random noise and domain-invariant features to simultaneously generate sensor data and corresponding labels
- Domain-Invariant Discriminator evaluates authenticity of generated sensor-label pairs using concatenated sensor and label information

3.6 Result Analysis

The comparative analysis of the five generative models reveals distinct behavioral patterns and capabilities in sensor data simulation.

3.6.1 RNN Performance

The model appears to converge to relatively constant values, indicating its inability to learn and reproduce the intricate patterns and dynamics inherent in the sensor measurements. This limitation makes RNN the poorest performer among all tested models, as it can only capture a limited number of simple patterns.

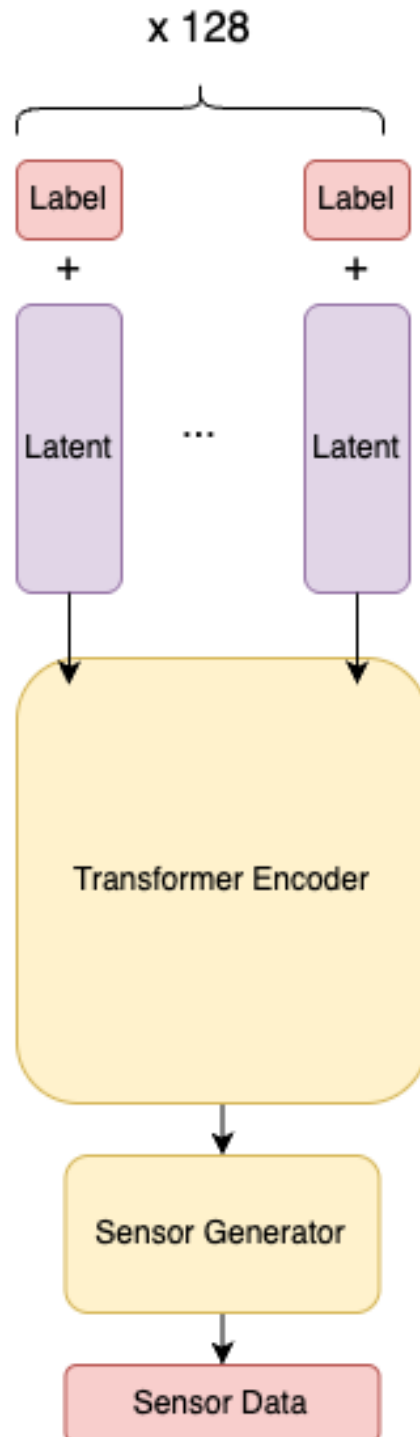


Figure 16: TGAN Generator 1

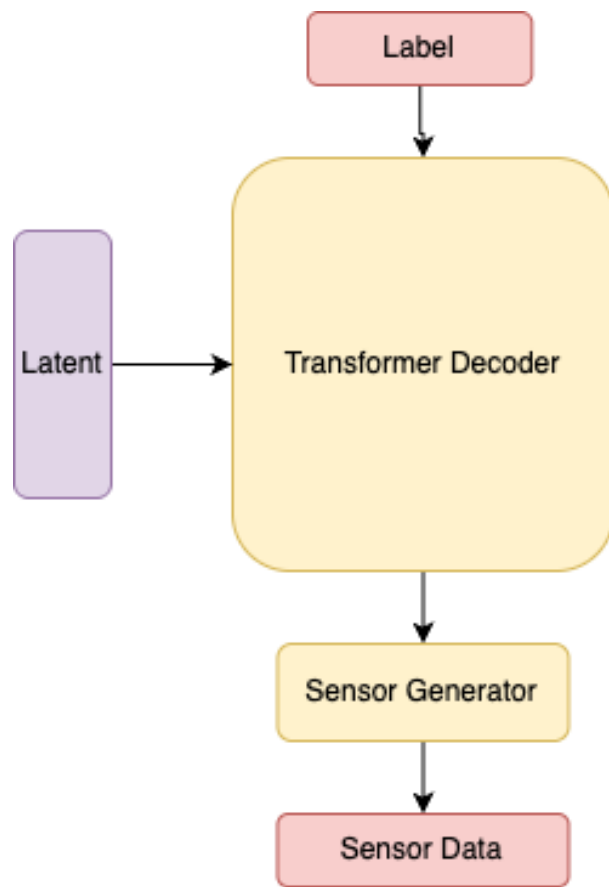


Figure 17: TGAN Generator 2

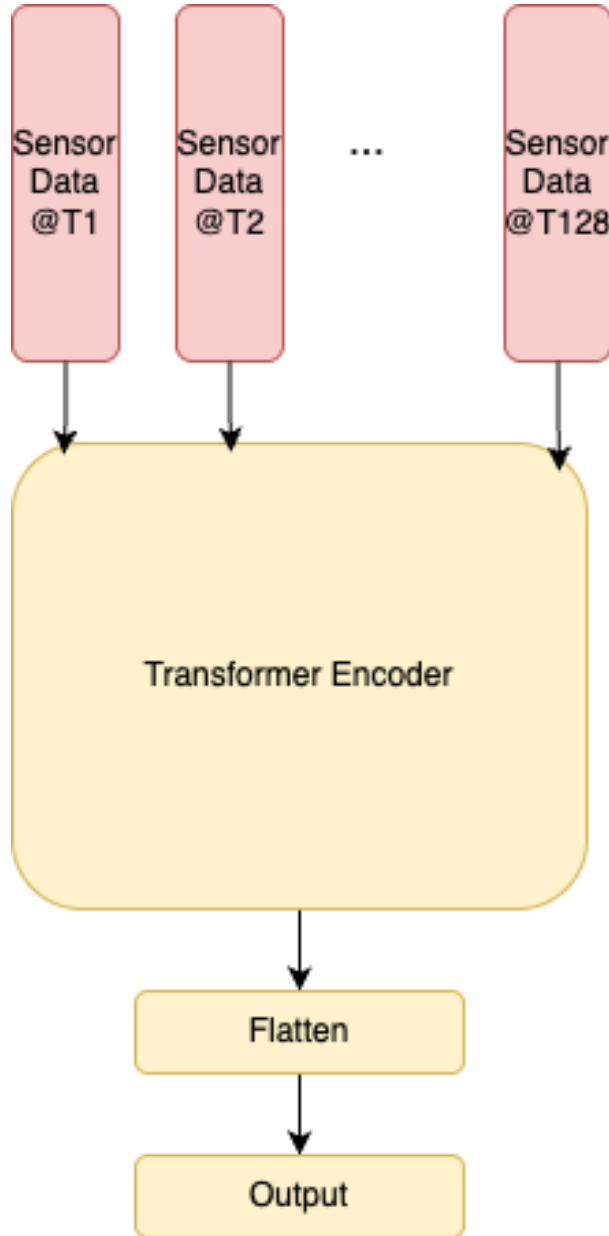


Figure 18: TGAN Discriminator

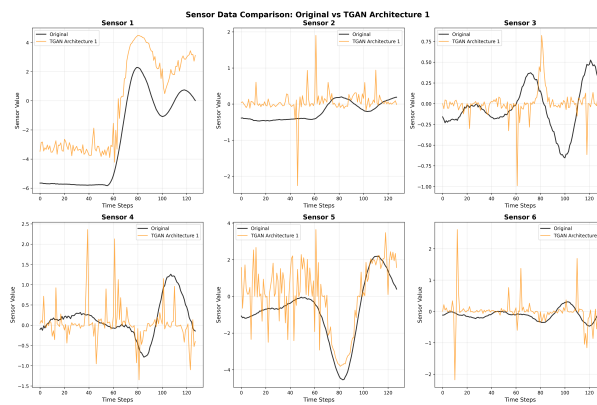


Figure 19: Data Comparison TGAN 1

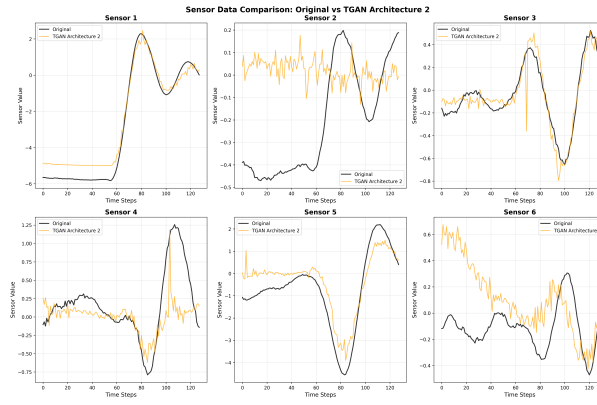


Figure 20: Data Comparison TGAN 2

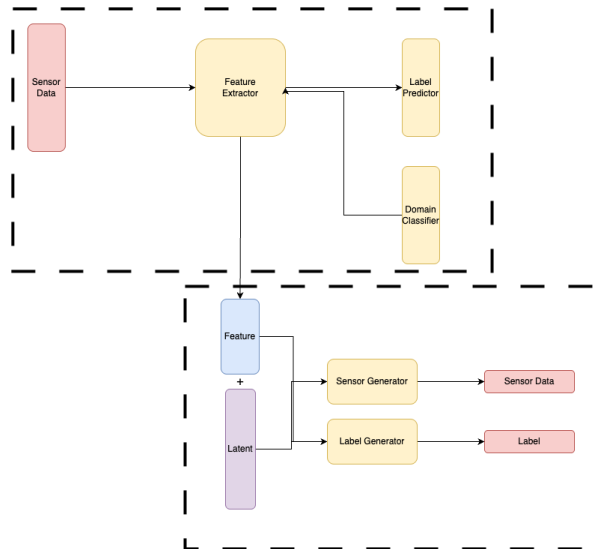


Figure 21: DANN Generator

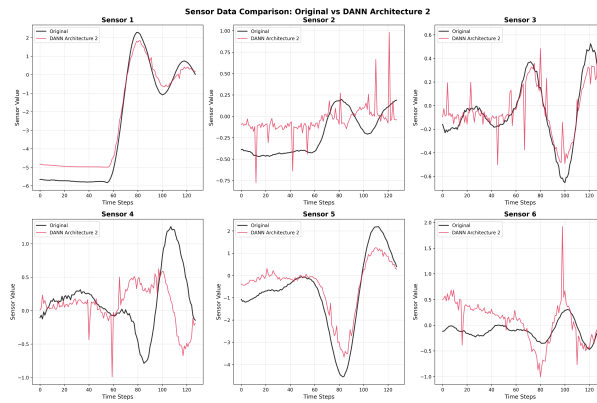


Figure 22: Data Comparison DANN

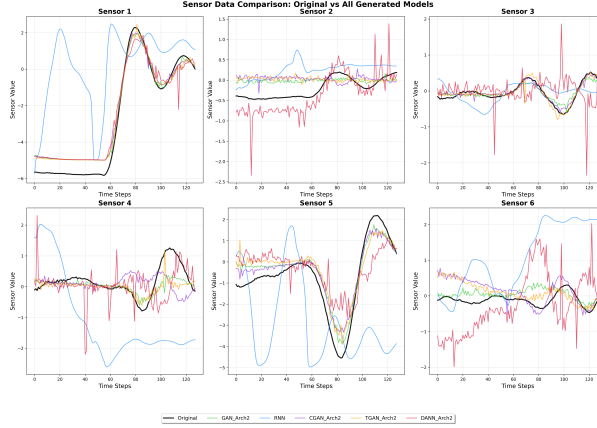


Figure 23: Data Comparison ALL Models

3.6.2 Transformer-based Model (TGAN)

TGAN successfully captures the underlying trends present in the original sensor data. The generated time series maintain statistical properties that closely match those of the original data across multiple sensors. While the generated sequences exhibit natural variability during trend evolution, the overall directional patterns and temporal dependencies are preserved. This capability to maintain trend coherence while incorporating realistic fluctuations makes TGAN the most effective model for sensor data generation among all tested approaches.

3.6.3 Computational Efficiency Considerations

Despite its superior performance in trend capture, TGAN comes with significant computational overhead. The model requires approximately four times the memory consumption compared to other models and exhibits training times that are approximately 15 times longer than conventional approaches. This substantial increase in computational requirements must be weighed against the improved quality of generated data when selecting an appropriate model for specific applications.

3.6.4 Summary

In conclusion, the Transformer-based GAN (TGAN) demonstrates the best performance in terms of data quality and temporal pattern preservation, making it the optimal choice for applications where data fidelity is paramount. However, for scenarios where computational efficiency is critical, the DANN model provides the best balance between performance and resource utilization, achieving comparable results with significantly reduced computational demands. The RNN model, while computationally lightweight, shows the poorest performance and is only suitable for applications with minimal pattern complexity requirements.

3.7 Dataset Expansion and Performance Improvement

The analysis of performance improvement resulting from dataset expansion with synthetic data is already included in each individual model's section.

4 Comparison of different models

(Jingyao Zhang)

4.0.1 Task and Data

This section presents a comprehensive comparison between the FCN, CNN, and LSTM models discussed in section 2, we compare different models of neural networks to obtain a good discriminative model to predicts whether a break does work or is defect given acceleration measurements and angular velocity in three dimensions and also the cars specifications as model, weight, velocity or deceleration average. The evaluation focuses on the challenges encountered with each architecture and the best results achieved during experimentation.

4.0.2 Observed Challenges:

- **FCN:** Using binary classification labels led to a loss of information, and the GAN-based data augmentation methods did not provide significant improvements for this architecture.
- **CNN:** Reduced task difficulty due to binary labels (accuracy decreases when multi-class labels are applied).Using generated data actually lowers model performance .
- **LSTM:** The main issue was the instability of the training process, making it challenging to achieve consistent results.

4.0.3 Best Model Configuration for Each Architecture:

- **FCN (Fully Convolutional Network):**
 - Used z-score standardization for input features
 - Binary classification labels
 - Dropout rate: 0.3
 - No early stopping (no patience parameter)
 - No random split for data partitioning
 - Batch size: 64
 - Optimizer: Adam with learning rate 0.001
 - L2 regularization (weight decay): 0.01
 - Loss function: CrossEntropyLoss with class weights
 - Number of epochs: 30

Accuracy of Best Model: 0.67

- **2D CNN (2D Convolutional Neural Network):**
 - Set random seedsand convert original multi-class labels to binary labels
 - Global Normalization
 - K-Fold Cross-Validation
 - Ensemble Learning: Soft voting and hard voting
 - Loss Function: CrossEntropyLoss
 - Activation Function: Rectified Linear Unit

- Batch size: 32
- Learning rate: $3e-3$
- epochs: 100
- No early stopping
- weight decay: $1e-4$

Accuracy of Best Model:

- **LSTM (Long Short-Term Memory Network):**

- Applied right label conversion (no normalization of inputs)
- Loss function: `nn.BCEWithLogitsLoss` with `pos_weight` ≈ 0.8 (calculated as `wb/bb`)
- Single hidden layer with 512 units
- Used hard-coded data augmentation or GAN-based augmentation
- Learning rate: 0.1 (10^{-1})

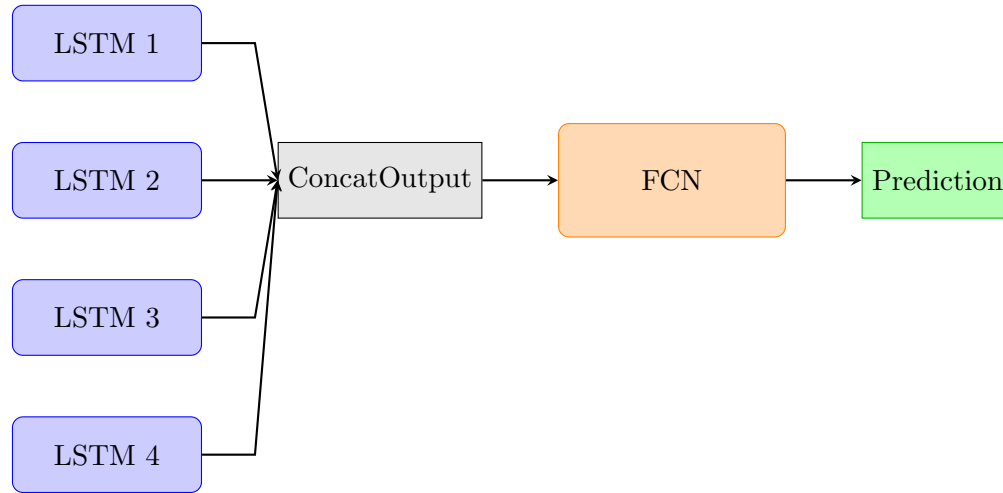
Accuracy of Best Model: 0.70

5 Additional Task 2: Merging Models

(Moritz Bauer)

We take 4 LSTM Networks, namely the model on the pure data with learning rate $lr = 10^{-2}$, and the models trained on hard coded data augmentation, GAN data and TGAN data.

We merged all predictions (as floats $p \in [0, 1]$) into one tensor of 4 predictions and pass it to a fully connected neural network with hidden dimension 32 to get a "merged" prediction.



This merged model is only classifying if a break is broken or not, but due to four well-trained models, we now can get a good prediction if a break is broken or not. We close this projects report with the confusion matrix of the merged model:

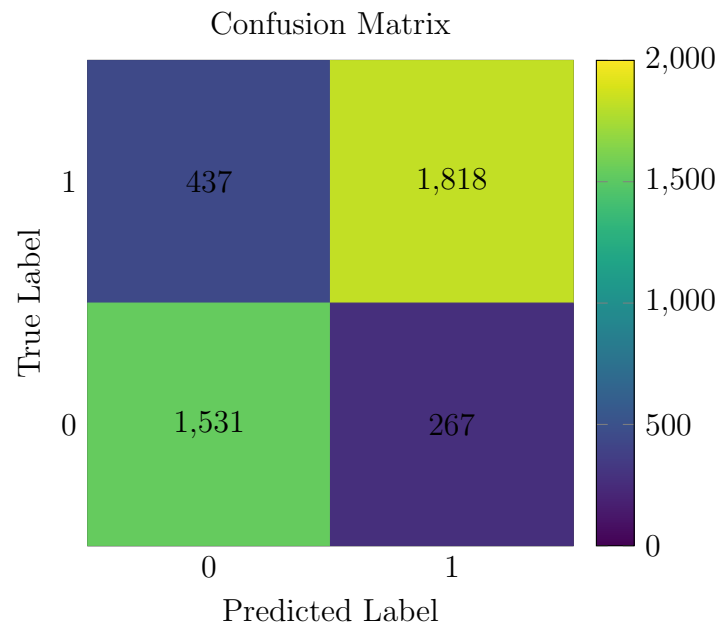


Figure 24: Right/False predictions per class 0/1 for working/broken breaks